
End-to-End Sequence-Graph Learning for User Behavior Modeling

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Modeling user behavior has become a core component in industrial applications
2 across a wide range of domains, including e-commerce, social media, health, fi-
3 nance, cybersecurity, streaming platforms, and online gaming. User behaviors
4 inherently consist of both sequential and relational components. Typically, they are
5 represented as sequences of events for each user, such as logging in, visiting pages,
6 viewing products, etc; forming a collection of sequential samples. In addition, other
7 aspects of user behavior form relational information, such as friendships, family
8 ties, or device sharing among users. Traditionally, practitioners modeling user
9 behaviors must choose between two paradigms: treating the data as independent
10 sequences via sequential models, or forcing all information into a graph representa-
11 tion and training a graph model on top of it. We propose that, instead of picking just
12 one paradigm, we represent inherently sequential behavior using a sequence model
13 and incorporate this into a graph model to capture the relational aspects of user be-
14 havior. To achieve this, we introduce an end-to-end training framework that jointly
15 optimizes both components. We demonstrate that this unified approach improves
16 model performance compared to relying on graph-only methods, sequence/text-
17 only methods, or two-stage sequence-graph models, where applicable.

18 1 Introduction

19 User behavior modeling has become a core component in many deployed industrial applications
20 across a wide range of domains, including e-commerce [1, 2], social media [3–5], health [6–8],
21 finance [9–12], cybersecurity [13, 14], streaming platforms [15–18], and online gaming [19–21]. The
22 behaviors of users inherently consist of sequential components, typically represented as an event
23 sequence for each user, and relational components, usually represented as a relational graph. For
24 example, in the case of e-commerce as illustrated in Figure 1, a user has an ordered event history,
25 such as logins, page visits, check-ins, product views, reviews, payments, and many other events. Each
26 user may also be connected to another user via user-to-user relationships, such as friendship, family
27 relations, sharing same addresses, or sharing device relations.

28 In health applications, a patient has an event history comprising doctor visits, height/weight measure-
29 ments, medical tests and their results, and medications taken; they may also have patient-to-patient
30 relationships such as family relations, genetic links, sharing same adverse reactions to certain med-
31 ications, or sharing same living environments. In finance applications, a customer’s event history
32 may contain events such as logging in to an app, checking a balance, withdrawing or depositing
33 money, purchasing a product, or making a payment. In addition, a customer may be related to other
34 customers via joint accounts, authorized users, or sharing devices.

35 For modeling user behaviors, practitioners across different areas are often forced to choose between
36 two main approaches: sequential representation or graph representation; due to the available models,

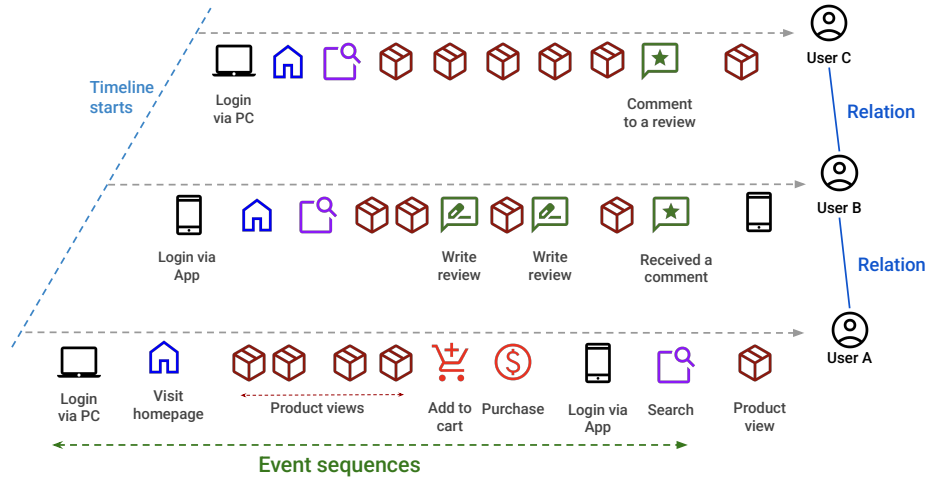


Figure 1: A simple example of user behavior data in e-commerce. Each user has a sequence of events, while simultaneously also having a static user-to-user relationship such as friendship, family relation, sharing same address, sharing same devices, etc.

37 algorithms, and tools being mostly built on top of only one of these modalities. However, picking
 38 one representation over another entails specific downsides. When representing user behavior using a
 39 sequential representation, as in personalized user event modeling [22, 23] or sequential recommenda-
 40 tion [24, 25], important user-to-user relationships are often left out. Conversely, when representing
 41 all aspects of behavior (including the sequential part) as a graph, everything must be forced into either
 42 a node or edge representation to be able to run graph models.

43 While this may work in some cases, there are multiple scenarios where certain behavioral aspects
 44 are not naturally representable as nodes or edges [26]. These include user events that do not involve
 45 interactions with other entities, such as intransitive actions (actions without objects) like "login,"
 46 "check-in," or "subscribe"; user status updates like "login successful," "payment approved/declined,"
 47 or "request declined"; and system-wide events like "website under maintenance" or "ongoing holiday
 48 promotion" in the e-commerce setting. Additionally, while the sequential information for each user
 49 contains dynamic, time-based, single-occurrence events, many user-to-user relationships, such as
 50 friendship or family relations, contain persistent, static information about the relationship rather than
 51 one-time event occurrences. This causes a problem when attempting to represent user behavior data
 52 solely using a graph representation, as the main types of graph representations are either static graphs
 53 or dynamic (temporal) graphs [27–29], the latter of which typically supports only event-based data.

54 There have been several related works on combining sequential and graph models, particularly in
 55 the field of sequential recommendation [30–35]. For example, Zhang et al. [33] define each sample
 56 as a sequence of items for a user, connected via sequential edges. They then add item-to-item
 57 collaborative edges denoting similarity, constructed by summarizing the user-item interaction matrix.
 58 While these sequential recommendation models contain both sequence and graph components, they
 59 are not suitable for the problem setting in this paper. Their formulation starts with a sequence of items
 60 for each user and then builds a graph representation from user-item relations. In contrast, our problem
 61 setting involves much more generic user behavior than simple user-item interactions. Additionally, our
 62 setup includes an inherent user-to-user relationship graph rather than a constructed user-item graph.

63 Inspired by the real-world user behavior modeling mentioned above, we propose a simple formal-
 64 ization that combines the flexibility of generic user events in personalized user event sequence
 65 formulations [22, 23] with user-to-user static graph formulations [36, 37]. We then propose BRIDGE,
 66 a simple end-to-end framework that combines sequential modeling methods, such as Transform-
 67 ers [38, 39], with static graph modeling methods by taking the outputs of the sequence module as the
 68 input to the graph module, which are then passed to a prediction head module. All components are
 69 linked together so that we can train end-to-end and propagate gradient signals across multiple modules.

70 Our contributions are summarized as follows:

- We propose a simple formulation for combining personalized user event sequence representations with static graph representations, motivated by the requirements of user behavior modeling and the limitations we identified in current formalizations.
- We introduce an end-to-end sequence-graph architecture that integrates a Transformer-based sequence module with static graph modules through a straightforward differentiable pipeline.
- We evaluate our framework on real-world user behavior datasets for two distinct tasks: relationship prediction and fraud detection. Our results demonstrate that this joint end-to-end model consistently outperforms graph-only and frozen two-stage alternatives, as well as sequence-only models where applicable. Additionally, we also include temporal-graph reformulations as a diagnostic to determine whether this specific user behavior modeling setup can be handled effectively by existing temporal graph models.

Our study focuses on a practical lesson: for user behavior data containing both sequential event histories and static relational context, explicitly combining the two structures is often more effective than forcing the data into a single representation (sequence-only or graph-only representation).

2 Related Work

This section reviews related work on problem formulations for sequence modeling and graph modeling. We provide additional, more complete related work in Appendix A.

User event sequence modeling, such as personalized event sequence [22, 23] and sequential recommendation [24, 25], models a set of N users $\mathcal{U} = \{u_1, \dots, u_N\}$, where each user maintains an independent sequence of events of length M_i , denoted as $S_i = [s_1^{(i)}, \dots, s_{M_i}^{(i)}]$. Each item $s_j^{(i)}$ represents an event for user u_i , drawn from an event set $\mathcal{S} = \{1, \dots, |\mathcal{S}|\}$. This formulation, however, does not consider cross-user relationships, rendering an incomplete observation of user behaviors.

Static graph modeling [36, 37], on the other hand, directly encodes user-to-user relationship via its node and edge formulation $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. However, it cannot encode sequential information of user events when modeling user behaviors.

Temporal graph models [40–42] incorporate time/sequential information into graphs by representing the graph as a sequence of timestamped edges $\mathcal{G} = [(u_1, v_1, t_1), (u_2, v_2, t_2), \dots]$, where u and v denote source and destination nodes at time t . However, its formulation does not allow the combination of event-based information and static relationship information. In addition, the formulation lacks the capability to encode naturally user events that do not involve interactions with other entities, such as intransitive actions, user status updates or system-wide events. While some temporal graph formulations can include node-wise events [43, 44], the concerns of lacking the capability to encode the full complexity of personalized user event sequences persist [26].

Two-stage sequence and graph integration. The personal and relational event sequence (PRES) formulation by Fathony et al. [26] augments the user event sequence formulation with relational information. Specifically, an event may be drawn from either a personal event set $\mathcal{P} = \{1, \dots, |\mathcal{P}|\}$ or a relational event set $\mathcal{R} = \{1, \dots, |\mathcal{R}|\}$. If an event is drawn from the relational event set, it must be accompanied by another user v . This formalization provides a way to combine sequential and relational information from the user event data. However, they also focused on event-based relationship information rather than static relational user-to-user information.

Fathony et al. [26] also proposed a method on integrating sequential and relational information that can be applied to our setting as well. Their methods uses sequence embeddings as graph node features. This validates the importance of both modalities, but its training pipeline separates sequence modeling from graph learning in a two-stage learning method. Our approach keeps the same practical motivation while using a single target-task objective to train the sequence and graph modules together.

Spatio-temporal graph models [45, 46] augment the static graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ with a time-synced feature sequence $[\mathbf{X}_1, \dots, \mathbf{X}_T]$, where $\mathbf{X}_t \in \mathbb{R}^{|\mathcal{V}| \times d}$, T is the time duration, and d is the feature dimension. The limitation is that the time dimension must be synced across all nodes. This fails for modeling user event sequences, as each user has independent events occurring at irregular intervals unsynced with other entities.

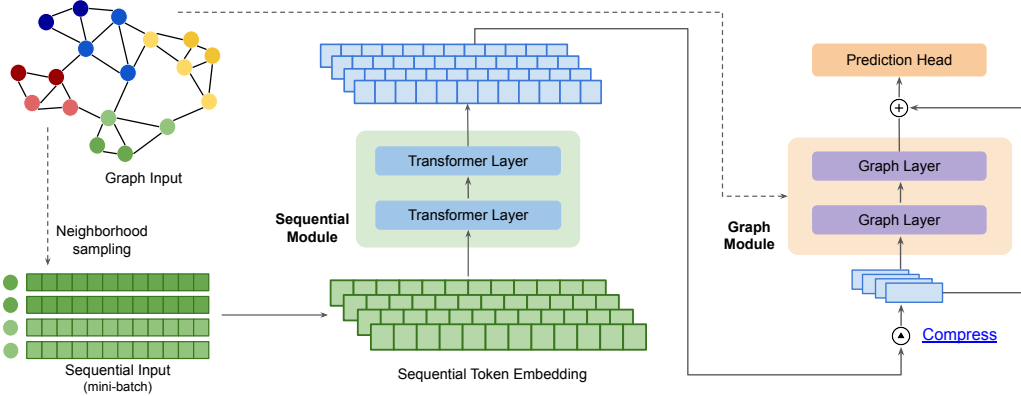


Figure 2: BRIDGE encodes each user’s event history with a sequence module, compresses the sequence representation into user-level node features, propagates information over the static user-relation graph, and optimizes the full pipeline end-to-end for the downstream task.

Graph in sequential recommendation task. There have also been efforts to augment the sequential recommendation task with graph models [30–35]. Specifically, they define each sample as a sequence of items for a user, connected via sequential edges. They then add item-to-item collaborative edges denoting similarity, constructed by summarizing the user-item interaction matrix. While their formulation contains both sequence and graph components, it still takes only sequential data (sequences of items) as input and constructs edges within the sequence, rather than modeling cross-user relationships.

3 Approach

3.1 Problem Formulation

Motivated by the problem of modeling user behaviors where each user carries sequential events and may have connections to another user via a static relationship, we define our formulation as follows:

Definition 3.1. Let $\mathcal{U} = \{u_1, \dots, u_N\}$ be a set of users. Each user u_i has an event history $S_i = [s_1^{(i)}, \dots, s_{M_i}^{(i)}]$, where $s_j^{(i)}$ is the j -th event in that user’s sequence. Users are also connected by a static relation graph $G = (\mathcal{U}, \mathcal{E})$, where an edge $(u_i, u_j) \in \mathcal{E}$ indicates a static user-to-user relation.

This formulation combines the personalized event sequence formulation [22, 23] with the static graph [36, 37] representation, taking advantage of both modalities. The formulation is also related to the PRES formulation [26], but differs in the representation of relational information. Our formulation adopts a static relational representation to fit the user behavior modeling we discussed above, whereas their formulation adopts an event-based representation of the relational information. The prediction task in this setting could be a relationship prediction (user-to-user link prediction), user classification, or any other user-level prediction problems.

3.2 Methodology

We introduce BRIDGE (Bimodal Representation Integration for Deep Graph-Sequence End-to-end Learning), a simple end-to-end architecture for the problem formulation in Definition 3.1. The architecture contains several sequential model layers, several static graph layers, modules for compression/aggregation, and a prediction head, as also shown in Figure 2.

We start with the sequential module of our architecture, which processes samples of a user’s event history. The event sequence for a user u_i is represented as a sequence of token embeddings $X_i \in \mathbb{R}^{M_i \times d}$, where M_i is the sequence length and d is the input embedding dimensions. The process of mapping an event to a token embedding is flexible and left to the practitioner’s discretion; however, for this notation, we assume that a single event is represented as an embedding vector of dimension

Table 1: Dataset statistics for relationship prediction (left) and Amazon fraud detection (right).

Relationship prediction				Fraud detection			
Dataset	# Users	# Rel.	# Events	Statistics	Movies	Elec.	Cloth.
Brightkite	58,228	428,156	4,702,710	# Total Users	139,138	254,064	185,986
Gowalla	196,591	1,900,654	6,442,289	# Labeled Users	50,268	114,840	67,988
A-Movies	139,138	2,807,368	1,932,230	# Fraud (Pos.)	13,912	7,456	1,658
A-Electronics	254,064	657,050	2,281,128	# Non-Fraud (Neg.)	36,356	107,384	66,330
A-Clothing	185,986	18,078	1,573,869	Fraud %	27.7	6.5	2.4

d. This sequence of token embeddings X_i is then passed through a series of sequential model layers, such as Transformer layers [38], resulting in a sequential representation $Z_i \in \mathbb{R}^{M_i \times d}$ for each user.

We then pass these sequential representations into a graph module, where we propagate messages among users. To be able to utilize the graph module, we must modify the construction of sample batches within the sequential module. Standard sequential models typically sample sequences from different entities at random to form a mini-batch. In contrast, our architecture samples a set of users and their corresponding sequences, then expands this set using neighborhood sampling, as is common in standard GNN algorithms. Consequently, a mini-batch entering the sequential module contains sequences that are interconnected via the underlying graph structure.

The sequential representation Z_i is passed to a compression module, $z_i = \text{Compress}(Z_i)$, where z_i is a vector of dimension d_c , i.e., $z_i \in \mathbb{R}^{d_c}$. This compression module may be a simple pooling mechanism (such as mean pooling) or any other technique designed to summarize a sequence into a single vector. The resulting compressed vector for each user is then treated as a trainable node feature for the graph module. This module performs message-passing operations among neighboring nodes via a series of graph convolutional layers. The final user representation h_i is computed by combining the output of the graph module with the compressed sequential representation z_i via a residual connection. Finally, this representation is passed to a prediction head to generate the task-specific output.

To train the model, we perform a forward pass through all the aforementioned components and compute an objective function based on the final output (details in Algorithm 1 in Appendix B). Since all operations across the modules are linked in a differentiable chain, we can perform end-to-end training of the entire system via backpropagation using standard automatic differentiation frameworks.

The key difference from two-stage training [26] is that BRIDGE is optimized as one differentiable system. The loss is computed from the final task prediction, and gradients flow through the graph module, compression module, and sequence module together. In our experiments, the graph module is instantiated with standard static graph model backbones such as GCN [36], GAT [37], and TFConv [47], resulting in three variations of our model: BRIDGE-GCN, BRIDGE-GAT, and BRIDGE-TFConv.

4 Experiments

We organize our experiments around user behavior modeling as described in Definition 3.1. Specifically, we ask two main research questions: (1) does incorporating both users' event sequences and their static user-to-user graph information improve results over models that use only graph or sequence information?, and (2) does incorporating them in an end-to-end fashion further improve results over two-stage sequence-then-graph pipelines? We conducted these experiments on two different tasks: relationship prediction and fraud detection.

Additionally, we provide a separate section for diagnosing the performance of temporal graph methods when the problem formulation in Definition 3.1 is converted into a temporal graph formulation. As noted in the motivation of our paper, modeling user behavior may involve both dynamic event-based data and static information on user-to-user relations. Section 4.3 describes whether this conversion of static information into timestamped temporal graph events is a good fit.

4.1 Relationship Prediction

Experimental Setup. We perform relationship prediction on the Brightkite and Gowalla datasets [48, 49], as well as Amazon reviews [50]. Brightkite and Gowalla are location-based social network-

Table 2: Relationship prediction performance across five datasets (higher is better). The main comparison uses methods aligned with the static-relation formulation: graph-only baselines, frozen two-stage sequence+graph baselines, and end-to-end BRIDGE variants. Best results are **bolded** and the runner-ups are underlined.

Method	Brightkite		Gowalla		Movies		Electronics		Clothing	
	MRR	H@5	MRR	H@5	MRR	H@5	MRR	H@5	MRR	H@5
<i>Graph Only (Static Graph)</i>										
GCN	67.4±1.9	82.3±1.3	77.8±0.6	89.4±0.3	76.4±0.5	89.6±0.5	48.9±3.9	66.6±3.3	17.3±1.5	23.3±1.9
GAT	65.4±1.6	80.1±1.0	78.1±0.6	89.3±0.3	76.6±0.7	90.8±0.6	44.7±0.7	62.5±0.7	16.2±2.3	18.0±2.1
TFConv	68.5±0.2	83.2±0.4	80.5±0.2	90.9±0.2	82.4±0.1	94.4±0.0	55.8±0.7	71.5±0.4	18.8±0.7	25.2±0.3
<i>Two-Stage (Static Graph + Sequence)</i>										
GCN + S	68.6±1.1	83.4±0.7	79.8±0.7	90.5±0.4	76.8±1.1	90.1±0.6	50.8±0.8	68.1±0.9	13.5±2.7	18.2±2.8
GAT + S	67.9±0.5	82.3±0.5	80.0±0.7	90.3±0.5	77.2±0.5	90.9±0.4	50.3±0.3	67.2±0.4	14.1±4.6	17.2±5.6
TFConv + S	72.7±0.3	86.3±0.1	82.5±0.5	92.0±0.3	83.6±0.1	95.1±0.1	58.4±0.2	73.5±0.1	17.7±1.1	24.2±1.1
BRIDGE										
BRIDGE-GCN	92.2±0.6	94.8±0.5	87.9±0.2	93.3±0.1	90.4±0.4	97.6±0.2	80.2±0.2	87.1±0.1	52.3±2.9	<u>72.5±2.2</u>
BRIDGE-GAT	78.8±0.4	89.5±0.1	<u>83.4±0.1</u>	<u>92.2±0.1</u>	89.1±0.0	<u>97.4±0.0</u>	75.4±0.3	<u>86.0±0.1</u>	63.7±0.8	75.2±0.2
BRIDGE-TFConv	78.4±0.7	89.3±0.4	82.9±0.1	92.1±0.1	88.6±0.4	97.2±0.1	75.1±0.2	85.7±0.2	<u>54.0±0.5</u>	68.0±0.7

ing platforms; each dataset contains location check-ins and friendship relationships among users. Each check-in, originally recorded in latitude and longitude, was converted into a Geohash-8 representation [51]. Example geohashes include 9v6kpmr1, gcpwkeq6, and u0yhxgm1, where nearby locations share common prefixes. For these datasets, each user possesses a sequence of geohash check-ins, which we further split into four tokens representing increasingly higher spatial resolutions. Both Brightkite and Gowalla treat friendship relations as static user-to-user properties without any timestamp information provided. For Amazon, we utilize the *Clothing*, *Electronics*, and *Movies* categories. Here, each user’s sequence consists of product IDs and ratings, and the static relation graph connects users who co-reviewed at least three products. Detailed data statistics are provided in Table 1; “A-” denotes Amazon categories. For more info about the datasets, see Fathony et al. [26].

Main Compared Methods. As our task contains both event sequence and static relational information, our main comparable baselines are models that can operate on the static graph formulation.

1. **Graph-Only Models:** GCN [36], GAT [37], and TFConv [47] operate on the static user-to-user relation graph, ignoring each user’s individual event sequences.
2. **Two-Stage Training:** Following Fathony et al. [26], GCN+S, GAT+S, and TFConv+S extend static graphs by incorporating sequence embeddings as node features. These embeddings are pre-trained using a bidirectional Transformer encoder via masked token prediction on user sequence tokens and remain frozen during graph training.
3. **BRIDGE:** Our model stacks a bidirectional Transformer sequence model with various GNN backbones, trained jointly in an end-to-end manner. Unlike the two-stage training approach, BRIDGE does not utilize frozen embeddings; instead, it optimizes the Transformer and GNN components simultaneously for the target task.

Evaluation Metrics. We use ranking-based metrics: MRR and Hits@ k , following prior relationship/link prediction benchmarks [52–54]. The main paper reports both MRR and Hits@5, while the full Hits@ k results are available in Appendix C. Each positive relation is ranked against 100 uniformly sampled non-neighbor users. All results are averaged over three independent runs.

Results. In Table 2, the results support the value of integrating sequential and relational information when modeling user behavior. Two-stage methods’ sequence embeddings generally improve over graph-only baselines, showing that users’ individual event sequences contain signals that are not captured by the static graph alone. However, training the graph and sequence model separately leaves substantial gains unrealized: BRIDGE variants obtain the best results across all five datasets, with especially significant improvements on Brightkite and the Amazon categories.

4.2 Fraud Detection

Experimental Setup. We evaluate fraud detection task on Amazon review data [50]. Each user’s sequence contains product IDs, ratings, and review text. Each review may receive “helpful” or

Table 3: Fraud detection performance across three Amazon categories. The main comparison uses text/sequence-only, static graph, frozen static graph+text, and end-to-end BRIDGE variants.

Method	Amazon-Movies		Amazon-Electronics		Amazon-Clothing	
	Max F1	PR AUC	Max F1	PR AUC	Max F1	PR AUC
<i>Static Graph Methods</i>						
GCN	43.8 $\pm$ 0.2	31.5 $\pm$ 1.3	11.9 $\pm$ 0.3	8.2 $\pm$ 0.3	5.4 $\pm$ 0.9	2.6 $\pm$ 0.3
GAT	43.8 $\pm$ 0.4	37.9 $\pm$ 1.7	12.2 $\pm$ 0.3	6.6 $\pm$ 0.3	5.7 $\pm$ 0.6	2.5 $\pm$ 0.1
TFConv	44.8 $\pm$ 0.3	41.2 $\pm$ 1.5	11.7 $\pm$ 0.3	8.2 $\pm$ 0.3	5.3 $\pm$ 0.6	2.6 $\pm$ 0.5
<i>Text Embedding Model</i>						
Sentence-BERT	75.7 $\pm$ 0.8	83.1 $\pm$ 1.0	46.4 $\pm$ 1.3	45.5 $\pm$ 2.5	36.1 $\pm$ 3.9	34.1 $\pm$ 4.8
<i>Static Graph Methods + Text Embedding</i>						
GAT	78.3 $\pm$ 1.0	85.9 $\pm$ 1.5	46.0 $\pm$ 1.5	44.6 $\pm$ 2.5	37.9 $\pm$ 1.5	35.8 $\pm$ 2.4
GCN	78.5 $\pm$ 1.0	86.0 $\pm$ 1.5	46.0 $\pm$ 1.3	45.0 $\pm$ 2.7	38.0 $\pm$ 1.6	35.9 $\pm$ 2.3
TFConv	78.8 $\pm$ 0.9	86.6 $\pm$ 1.2	46.7 $\pm$ 0.9	45.1 $\pm$ 2.2	37.8 $\pm$ 1.9	35.4 $\pm$ 1.7
BRIDGE						
BRIDGE-GCN	80.1 $\pm$ 1.0	87.8 $\pm$ 1.0	48.9 $\pm$ 1.2	48.3 $\pm$ 2.6	38.4 $\pm$ 2.2	36.6 $\pm$ 1.8
BRIDGE-GAT	80.3 $\pm$ 0.8	88.0 $\pm$ 0.7	47.3 $\pm$ 2.6	46.1 $\pm$ 3.1	37.9 $\pm$ 1.4	35.6 $\pm$ 2.1
BRIDGE-TFConv	80.1 $\pm$ 0.8	87.6 $\pm$ 0.9	50.0 $\pm$ 2.7	50.1 $\pm$ 3.3	37.3 $\pm$ 0.2	34.2 $\pm$ 1.2

“unhelpful” votes from other users. Following Dou et al. [55] and Fathony et al. [26], we use this helpfulness information as a proxy for fraudulent review incidents. Specifically, we label a user with at least 5 reviews as fraudulent if more than 70% of their reviews are marked unhelpful. Static relation edges connect users who co-reviewed at least three products. Dataset statistics are in Table 1.

Main Compared Methods. As in relationship prediction, the main comparison uses models aligned with the static-relation formulation. We evaluate graph-only models, Sentence-BERT as a text/sequence-only comparator, static graph + text embedding models as frozen two-stage comparators, and end-to-end BRIDGE variants.

1. **Graph-Only Models.** GCN, GAT, and TFConv operate on a static co-review graph without additional inputs.
2. **Text Embedding Model.** We encode each review with Sentence-BERT [56] to obtain a sequence of text embeddings per user. For classification, we then average the embeddings to form a single user representation, which is passed to a classifier.
3. **Static Graph + Text Embedding.** We incorporate the averaged text embeddings per user into static graph models as additional node features. This strategy is similar to the two-stage modeling in the previous task.
4. **BRIDGE:** Our approach takes sequence of Sentence-BERT embeddings of the review texts combined with a learnable rating embedding as input, passing them through the sequential module of our architecture. The rating embedding, sequential module, and graph module are trained jointly for fraud detection, allowing the model to integrate textual, sequential, and relational signals in an end-to-end manner.

Evaluation Metrics. For the fraud detection task, we report two classification metrics: **Max F1** (the best F1 metric across thresholds) and **PR-AUC** (area under the precision–recall curve).

Results. Table 3 shows that review text is highly informative for fraud detection, while the co-review patterns encoded in the static graph are less useful. Graph-only models underperform substantially, while the Sentence-BERT baseline perform quite well. Static graph + text embeddings improve over the text-only model on most metrics, indicating that the co-review relations add additional signals to the model. However, the strongest results come from training the text and sequence representation with the graph module jointly. The best results across all datasets and metrics comes from BRIDGE variants, indicating the benefit of the end-to-end learning approach.

4.3 Temporal Graph Diagnostic

Temporal Graph methods [40–42] are popular methodologies for modeling user behavior that consists of interaction events. Their formulation focuses on modeling the dynamic evolution of edges (and

Table 4: Relationship prediction temporal-graph diagnostic. Static graph methods use the given static user-relation graph directly. Temporal graph methods require converting user histories and static relations into a timestamped edge stream. Higher is better.

Method	Brightkite		Gowalla		Movies		Electronics		Clothing	
	MRR	H@5	MRR	H@5	MRR	H@5	MRR	H@5	MRR	H@5
<i>Static Graph Methods</i>										
GCN	67.4 $\pm$ 1.9	82.3 $\pm$ 1.3	77.8 $\pm$ 0.6	89.4 $\pm$ 0.3	76.4 $\pm$ 0.5	89.6 $\pm$ 0.5	48.9 $\pm$ 3.9	66.6 $\pm$ 3.3	17.3 $\pm$ 1.5	23.3 $\pm$ 1.9
GAT	65.4 $\pm$ 1.6	80.1 $\pm$ 1.0	78.1 $\pm$ 0.6	89.3 $\pm$ 0.3	76.6 $\pm$ 0.7	90.8 $\pm$ 0.6	44.7 $\pm$ 0.7	62.5 $\pm$ 0.7	16.2 $\pm$ 2.3	18.0 $\pm$ 2.1
TFCnv	68.5 $\pm$ 0.2	83.2 $\pm$ 0.4	80.5 $\pm$ 0.2	90.9 $\pm$ 0.2	82.4 $\pm$ 0.1	94.4 $\pm$ 0.0	55.8 $\pm$ 0.7	71.5 $\pm$ 0.4	18.8 $\pm$ 0.7	25.2 $\pm$ 0.3
<i>Temporal Graph Methods After Conversion</i>										
DyRep	26.4 $\pm$ 3.7	37.5 $\pm$ 5.5	27.8 $\pm$ 2.1	37.8 $\pm$ 3.3	41.6 $\pm$ 3.8	58.4 $\pm$ 3.2	17.0 $\pm$ 1.9	23.7 $\pm$ 2.1	13.3 $\pm$ 0.8	18.9 $\pm$ 1.3
TGN	38.7 $\pm$ 1.6	53.9 $\pm$ 1.3	44.8 $\pm$ 2.7	59.3 $\pm$ 3.4	59.1 $\pm$ 3.2	77.6 $\pm$ 2.1	33.8 $\pm$ 1.5	46.5 $\pm$ 2.1	14.8 $\pm$ 1.4	20.8 $\pm$ 3.0
TCL	20.4 $\pm$ 1.0	29.1 $\pm$ 1.5	20.5 $\pm$ 1.7	27.5 $\pm$ 2.5	72.2 $\pm$ 1.4	87.6 $\pm$ 0.8	34.9 $\pm$ 0.2	47.3 $\pm$ 0.1	14.2 $\pm$ 0.5	20.5 $\pm$ 1.3
NAT	68.2 $\pm$ 7.8	77.0 $\pm$ 5.9	59.1 $\pm$ 13.0	68.0 $\pm$ 11.8	78.8 $\pm$ 4.0	90.7 $\pm$ 2.7	62.8 $\pm$ 8.0	70.4 $\pm$ 6.4	30.6 $\pm$ 7.9	36.8 $\pm$ 7.2
GraphMixer	20.6 $\pm$ 0.5	31.0 $\pm$ 0.4	24.9 $\pm$ 0.5	35.5 $\pm$ 0.7	55.0 $\pm$ 0.8	72.5 $\pm$ 0.8	23.1 $\pm$ 0.5	34.3 $\pm$ 0.5	12.9 $\pm$ 0.3	17.7 $\pm$ 0.4
TPNet	45.0 $\pm$ 0.6	53.6 $\pm$ 0.4	51.4 $\pm$ 1.0	62.9 $\pm$ 1.3	42.5 $\pm$ 0.9	61.6 $\pm$ 1.2	22.9 $\pm$ 0.2	34.7 $\pm$ 0.3	11.4 $\pm$ 0.6	14.8 $\pm$ 1.1
TNCN	53.8 $\pm$ 0.5	65.9 $\pm$ 1.0	51.8 $\pm$ 0.7	64.0 $\pm$ 0.8	20.8 $\pm$ 1.2	28.6 $\pm$ 2.1	45.4 $\pm$ 1.1	55.8 $\pm$ 0.9	31.7 $\pm$ 1.8	39.6 $\pm$ 2.1

Table 5: Fraud detection temporal-graph diagnostic. Static graph methods use averaged Sentence-BERT review embeddings as node features. Temporal graph methods receive review text and ratings through the converted timestamped edge stream. Higher is better.

Method	Amazon-Movies		Amazon-Electronics		Amazon-Clothing	
	Max F1	PR AUC	Max F1	PR AUC	Max F1	PR AUC
<i>Static Graph Methods + Text Embedding</i>						
GAT	78.3 $\pm$ 1.0	85.9 $\pm$ 1.5	46.0 $\pm$ 1.5	44.6 $\pm$ 2.5	37.9 $\pm$ 1.5	35.8 $\pm$ 2.4
GCN	78.5 $\pm$ 1.0	86.0 $\pm$ 1.5	46.0 $\pm$ 1.3	45.0 $\pm$ 2.7	38.0 $\pm$ 1.6	35.9 $\pm$ 2.3
TFCnv	78.8 $\pm$ 0.9	86.6 $\pm$ 1.2	46.7 $\pm$ 0.9	45.1 $\pm$ 2.2	37.8 $\pm$ 1.9	35.4 $\pm$ 1.7
<i>Temporal Graph Methods After Conversion</i>						
DyRep	74.7 $\pm$ 0.7	82.3 $\pm$ 0.4	46.5 $\pm$ 0.3	46.3 $\pm$ 1.0	34.0 $\pm$ 2.8	33.2 $\pm$ 3.4
GraphMixer	69.1 $\pm$ 0.7	74.4 $\pm$ 0.3	32.7 $\pm$ 1.2	26.0 $\pm$ 1.3	19.8 $\pm$ 3.0	15.6 $\pm$ 3.6
NAT	51.2 $\pm$ 0.9	40.9 $\pm$ 0.5	18.2 $\pm$ 0.1	10.6 $\pm$ 0.2	8.8 $\pm$ 0.1	4.1 $\pm$ 0.3
TCL	74.8 $\pm$ 0.6	82.5 $\pm$ 0.1	32.6 $\pm$ 17.5	45.5 $\pm$ 6.8	14.9 $\pm$ 14.8	27.5 $\pm$ 24.0
TGN	75.5 $\pm$ 0.6	82.9 $\pm$ 0.3	46.8 $\pm$ 0.7	46.9 $\pm$ 1.3	35.0 $\pm$ 2.5	33.5 $\pm$ 2.9
TPNet	72.3 $\pm$ 0.8	79.2 $\pm$ 0.5	40.1 $\pm$ 0.8	35.8 $\pm$ 1.0	20.2 $\pm$ 1.9	15.6 $\pm$ 0.9

nodes) over time by denoting the graph as sequences of timestamped edges [40, 57–59]. This creates a mismatch when we want to apply these techniques to model user behavior that contains both dynamic personal user events and static user-to-user relationships, as their formulation does not support static relational information. Therefore, to apply temporal graph models, we are required to convert the original problem into an interaction event stream formulation.

For the relationship prediction task, users’ individual events are converted into timestamped edges between the user node and the event node, while static relationship edges are assigned synthetic timestamps. For the fraud detection task, each review becomes a timestamped user-item edge, static co-review edges receive synthetic timestamps, review text is attached as per-edge message features, ratings are added as edge features, and nodes are initialized from pooled Sentence-BERT embeddings. The results of these experiments serve as diagnostic rather than primary baseline comparisons: they test whether this conversion to a temporal-graph formulation is a viable substitute for the native formulation (Definition 3.1) of user behavior modeling with sequential user event and static user-to-user relations.

Table 4 shows that temporal graph methods with this conversion are inconsistent in the relationship prediction tasks. Some temporal models are competitive on individual Amazon categories, but they do not reliably outperform the simpler static graph baselines across datasets. Table 5 shows the same pattern for fraud detection: even with review text, ratings, and initialized node embeddings, temporal graph methods do not consistently improve over static graph + text baselines. These results do not imply that temporal graph methods are weak in their intended setting; rather, they show that forcing static relations and individual user event sequences into a timestamped edge stream introduces artificial modeling choices without producing a stronger representation for our tasks.

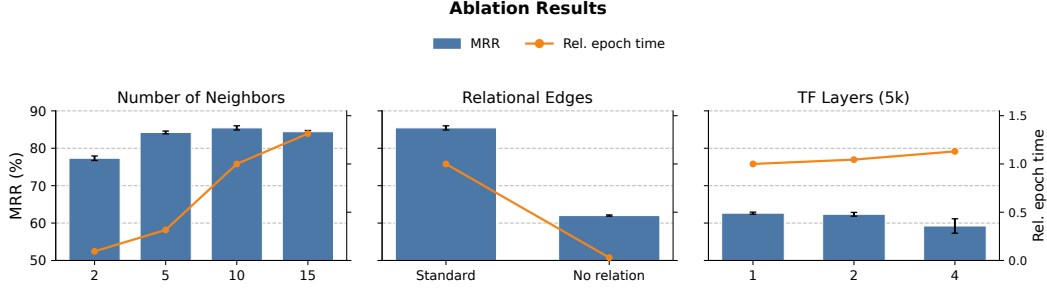


Figure 3: **Ablations on Brightkite** Bars indicate test MRR (%) and the orange line indicates relative epoch time (right axis) across (i) sampled neighbors, (ii) No-relation ablation, and (iii) sequence module depth. Runtime is normalized within each sweep to its reference setting.

5 Practical Design Tradeoffs

We ablate three practical choices in BRIDGE on Brightkite dataset: neighbor fanout and sequence encoder depth, and the graph module. Figure 3 reports MRR and relative epoch time; full values are in Appendix D.

Number of Neighbors From the first panel of Figure 3, reducing the neighbor sampling size from 10 to 5 or 2 lowers the runtime substantially, with only a moderate drop in MRR. This shows that BRIDGE remains usable under tighter compute budgets, but also that BRIDGE benefits from access to nontrivial relational context. Overall, 10 or 15 neighbors offer the strongest performance, while 2–5 neighbors provide a cheaper but weaker operating point when runtime is the primary constraint. Exact performance and epoch time are reported in Appendix D.

No-relation ablation. We next evaluate a no-relation ablation that removes all edges from the message-passing graph while keeping the same sequence module, prediction objective, and end-to-end training procedure. This is stricter than setting the neighborhood fanout to zero: rather than modifying only the sampler, we remove the relational pathway itself while preserving the same train/validation/test supervision. The resulting comparison isolates the contribution of relational propagation beyond joint sequence modeling alone. Compared with the full BRIDGE model, the no-relation variant performs worse, showing that BRIDGE’s gains do not come only from learning stronger sequence representations; they also depend on propagating information over the relational graph.

Transformer depth. The third panel of Figure 3 examines the effect of sequence-module depth. Because deeper sequence encoders incur higher computational cost, we conduct this ablation on a 5,000-user induced subgraph of Brightkite dataset.

Increasing the number of Transformer layers from 1 to 2 yields nearly identical MRR, while increasing the depth from 2 to 4 slightly degrades performance. At the same time, relative epoch time increases modestly with depth. These results suggest that, on the ablation subgraph, a shallow sequence encoder already captures most of the useful sequential signal once combined with graph context. Accordingly, we adopt a 1-layer Transformer encoder as the default configuration in the main experiments, as it provides the best trade-off between performance and efficiency.

6 Conclusion

We presented BRIDGE, a simple end-to-end architecture for user behavior modeling with sequential event histories and static user-to-user relation graphs. Across relationship prediction and fraud detection, jointly training the sequence and graph module improves over graph-only models and two-stage sequence-graph training alternatives. Temporal-graph reformulations require synthetic conversion choices and do not reliably close the gap. These results support a practical message: when the data naturally contains both event histories and static relational context, preserving both structures is often more effective than forcing the problem into only one representation.

References

- [1] Meng Zeng, Hong Cao, Min Chen, and Yujie Li. User behaviour modeling, recommendations, and purchase prediction during shopping festivals. *Electronic Markets*, 29(2):205–217, 2019.
- [2] Zhicheng He, Weiwen Liu, Wei Guo, Jiarui Qin, Yingxue Zhang, Yaochen Hu, and Ruiming Tang. A survey on user behavior modeling in recommender systems. *arXiv preprint arXiv:2302.11087*, 2023.
- [3] Ahmed Abdel-Hafez and Yanchun Xu. A survey of user modelling in social media websites. *Computer and Information Science*, 6(4):59–71, 2013.
- [4] Guangyuan Piao and John G Breslin. Inferring user interests in microblogging social networks: A survey. *User Modeling and User-Adapted Interaction*, 28(3):277–329, 2018.
- [5] Qixiang Fang, Zhihan Zhou, Francesco Barbieri, Yozen Liu, Leonardo Neves, Dong Nguyen, Daniel L Oberski, Maarten W Bos, and Ron Dotsch. General-purpose user modeling with behavioral logs: A snapchat case study. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2431–2436, 2023.
- [6] Yikuan Li, Shishir Rao, José Roberto Ayala Solares, Abdelaali Hassaine, Rema Ramakrishnan, Dexter Canoy, Yajie Zhu, Kazem Rahimi, and Gholamreza Salimi-Khorshidi. Behrt: transformer for electronic health records. *Scientific reports*, 10(1):7155, 2020.
- [7] Jeong Min Lee and Milos Hauskrecht. Personalized event prediction for electronic health records. *Artificial intelligence in medicine*, 143:102620, 2023.
- [8] Ethan Steinberg, Jason Alan Fries, Yizhe Xu, and Nigam Shah. Motor: A time-to-event foundation model for structured medical records. In *The Twelfth International Conference on Learning Representations*, 2023.
- [9] Akib Mashrur, Wei Luo, Nayyar A Zaidi, and Antonio Robles-Kelly. Machine learning for financial risk management: A survey. *IEEE Access*, 8:203203–203223, 2020.
- [10] Ahmet Murat Ozbayoglu, Mehmet Ugur Gudelek, and Omer Berat Sezer. Deep learning for financial applications: A survey. *Applied Soft Computing*, 93:106384, 2020.
- [11] S Navid Hojaji, Mahmood Yahyazadehfar, and Bahareh Abedin. Machine learning in behavioral finance: A systematic literature review. *The Journal of Financial Data Science*, 4(3):129–146, 2022.
- [12] Luisa Hernandez Aros, Ximena Bustamante Molano, Francisco Gutierrez-Portela, and Juan Jose Moreno Hernandez. Financial fraud detection through the application of machine learning techniques: a literature review. *Palgrave Communications*, 11(1):1–15, 2024.
- [13] Ahmad H Lashkari, Meng Chen, and Ali A Ghorbani. A survey on user profiling model for anomaly detection in cyberspace. *Journal of Information Security and Applications*, 34:38–56, 2017.
- [14] Gang Wang, Xinyang Zhang, Shaomei Tang, Christo Wilson, Haitao Zheng, and Ben Y Zhao. Clickstream user behavior models. *ACM Transactions on the Web (TWEB)*, 11(4):1–37, 2017.
- [15] Daniel Gayo-Avello. A survey on session detection methods in query logs and a proposal for future evaluation. *Information Sciences*, 179(12):1822–1843, 2009.
- [16] Xavier Amatriain and Justin Basilico. Big & personal: data and models behind netflix recommendations. *ACM SIGKDD Explorations Newsletter*, 14(2):37–48, 2013.
- [17] Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, pages 191–198, 2016.
- [18] Yan Zhao, Shoujin Wang, Yan Wang, and Hongwei Liu. Mbsrs: A multi-behavior streaming recommender system. *Information Sciences*, 631:1–17, 2023.

- [19] Yilei Zeng. How human centered ai will contribute towards intelligent gaming systems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 15742–15743, 2021.
- [20] Si Chen, Yuqiu Qian, Hui Li, and Chen Lin. Sequential recommendation in online games with multiple sequences, tasks and user levels. In *Proceedings of the 17th International Symposium on Spatial and Temporal Databases*, pages 130–140, 2021.
- [21] Junming Yin, Yue Feng, and Yong Liu. Modeling behavioral dynamics in digital content consumption: An attention-based neural point process approach with applications in video games. *Marketing Science*, 44(1):220–239, 2025.
- [22] Benjamin Letham, Cynthia Rudin, and David Madigan. Sequential event prediction. *Machine learning*, 93(2):357–380, 2013.
- [23] Alex Boyd, Robert Bamler, Stephan Mandt, and Padhraic Smyth. User-dependent neural sequence models for continuous-time event data. *Advances in Neural Information Processing Systems*, 33:21488–21499, 2020.
- [24] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. Session-based recommendations with recurrent neural networks. In Yoshua Bengio and Yann LeCun, editors, *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016.
- [25] Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*, pages 197–206. IEEE, 2018.
- [26] Rizal Fathony, Igor Melnyk, Owen Reinert, Nam H Nguyen, Daniele Rosa, and C Bayan Bruss. Integrating sequential and relational modeling for user events: Datasets and prediction tasks. In *The Fourth Learning on Graphs Conference*, 2025.
- [27] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. Temporal graph networks for deep learning on dynamic graphs. In *ICML 2020 Workshop on Graph Representation Learning*, 2020.
- [28] da Xu, chuanwei ruan, evren korpeoglu, sushant kumar, and kannan achan. Inductive representation learning on temporal graphs. In *International Conference on Learning Representations (ICLR)*, 2020.
- [29] Rakshit Trivedi, Mehrdad Farajtabar, Prasenjeet Biswal, and Hongyuan Zha. Dyrep: Learning representations over dynamic graphs. In *International conference on learning representations*, 2019.
- [30] Jingyue Gao, Yang Lin, Yasha Wang, Xiting Wang, Zhao Yang, Yuanduo He, and Xu Chu. Set-sequence-graph: A multi-view approach towards exploiting reviews for recommendation. In *Proceedings of the 29th ACM international conference on information & knowledge management*, pages 395–404, 2020.
- [31] Jianxin Chang, Chen Gao, Yu Zheng, Yiqun Hui, Yanan Niu, Yang Song, Depeng Jin, and Yong Li. Sequential recommendation with graph neural networks. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*, pages 378–387, 2021.
- [32] Zeren Shui, Ge Liu, Anoop Deoras, and George Karypis. Sequence-graph duality: Unifying user modeling with self-attention for sequential recommendation, 2022.
- [33] Mengqi Zhang, Shu Wu, Xueli Yu, Qiang Liu, and Liang Wang. Dynamic Graph Neural Networks for Sequential Recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 35(05):4741–4753, May 2023. ISSN 1041-4347. doi: 10.1109/TKDE.2022.3151618. Publisher: IEEE Computer Society.
- [34] Vladimir Baikalov and Evgeny Frolov. End-to-end graph-sequential representation learning for accurate recommendations. In *Companion Proceedings of the ACM Web Conference 2024*, pages 501–504, 2024.

- [35] Yuwei Cao, Liangwei Yang, Zhiwei Liu, Yuqing Liu, Chen Wang, Yueqing Liang, Hao Peng, and Philip S Yu. Graph-sequential alignment and uniformity: Toward enhanced recommendation systems. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 888–892, 2025.
- [36] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.
- [37] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph Attention Networks, February 2018. arXiv:1710.10903 [stat].
- [38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is All you Need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [39] Bryan Lim, Sercan Ö. Arik, Nicolas Loeff, and Tomas Pfister. Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, October 2021. ISSN 0169-2070. doi: 10.1016/j.ijforecast.2021.03.012.
- [40] Yuhong Luo and Pan Li. Neighborhood-aware scalable temporal network representation learning. In *Learning on Graphs Conference*, pages 1–1. PMLR, 2022.
- [41] Le Yu, Leilei Sun, Bowen Du, and Weifeng Lv. Towards Better Dynamic Graph Learning: New Architecture and Unified Library, October 2023. arXiv:2303.13047 [cs].
- [42] Alessio Gravina, Giulio Lovisotto, Claudio Gallicchio, Davide Bacciu, and Claas Grohnfeldt. Long range propagation on continuous-time dynamic graphs. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 16206–16225. PMLR, 21–27 Jul 2024.
- [43] Giang Hoang Nguyen, John Boaz Lee, Ryan A Rossi, Nesreen K Ahmed, Eunye Koh, and Sungchul Kim. Continuous-time dynamic network embeddings. In *Companion Proceedings of the The Web Conference 2018*, pages 969–976, 2018.
- [44] Seyed Mehran Kazemi, Rishab Goel, Kshitij Jain, Ivan Kobyzev, Akshay Sethi, Peter Forsyth, and Pascal Poupart. Representation learning for dynamic graphs: A survey. *Journal of Machine Learning Research*, 21(70):1–73, 2020.
- [45] Bing Yu, Haoteng Yin, and Zhanxing Zhu. Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*, pages 3634–3640, July 2018. doi: 10.24963/ijcai.2018/505. arXiv:1709.04875 [cs].
- [46] Hongjun Wang, Jiyuan Chen, Tong Pan, Zheng Dong, Lingyu Zhang, Renhe Jiang, and Xuan Song. STGformer: Efficient Spatiotemporal Graph Transformer for Traffic Forecasting, October 2024. arXiv:2410.00385 [cs] version: 1.
- [47] Yunsheng Shi, Zhengjie Huang, Shikun Feng, Hui Zhong, Wenjing Wang, and Yu Sun. Masked label prediction: Unified message passing model for semi-supervised classification. In Zhi-Hua Zhou, editor, *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21*, pages 1548–1554. International Joint Conferences on Artificial Intelligence Organization, 8 2021. doi: 10.24963/ijcai.2021/214. Main Track.
- [48] Eunjoon Cho, Seth A. Myers, and Jure Leskovec. Friendship and mobility: user movement in location-based social networks. In *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD ’11*, page 1082–1090, New York, NY, USA, 2011. Association for Computing Machinery. ISBN 9781450308137. doi: 10.1145/2020408.2020579.
- [49] Jure Leskovec and Andrej Krevl. SNAP Datasets: Stanford large network dataset collection. <http://snap.stanford.edu/data>, June 2014.

- [50] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton van den Hengel. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '15, page 43–52, New York, NY, USA, 2015. Association for Computing Machinery. ISBN 9781450336215. doi: 10.1145/2766462.2767755.
- [51] Gustavo Niemeyer. Geohash, 2008.
- [52] Julia Gastinger, Shenyang Huang, Mikhail Galkin, Erfan Loghmani, Ali Parviz, Farimah Poursafaei, Jacob Danovitch, Emanuele Rossi, Ioannis Koutis, Heiner Stuckenschmidt, Reihaneh Rabbany, and Guillaume Rabusseau. Tgb 2.0: A benchmark for learning on temporal knowledge graphs and heterogeneous graphs. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 140199–140229. Curran Associates, Inc., 2024. doi: 10.52202/079017-4450.
- [53] Shenyang Huang, Farimah Poursafaei, Jacob Danovitch, Matthias Fey, Weihua Hu, Emanuele Rossi, Jure Leskovec, Michael M. Bronstein, Guillaume Rabusseau, and Reihaneh Rabbany. Temporal graph benchmark for machine learning on temporal graphs. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.
- [54] Lu Yi, Jie Peng, Yanping Zheng, Fengran Mo, Zhewei Wei, Yuhang Ye, Yue Zixuan, and Zengfeng Huang. TGB-seq benchmark: Challenging temporal GNNs with complex sequential dynamics. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [55] Yingdong Dou, Zhiwei Liu, Li Sun, Yutong Deng, Hao Peng, and Philip S Yu. Enhancing graph neural network-based fraud detectors against camouflaged fraudsters. In *Proceedings of the 29th ACM international conference on information & knowledge management*, pages 315–324, 2020.
- [56] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2019. URL <https://arxiv.org/abs/1908.10084>.
- [57] Xiaohui Zhang, Yanbo Wang, Xiyuan Wang, and Muhan Zhang. Efficient neural common neighbor for temporal graph link prediction, 2024. URL <https://arxiv.org/abs/2406.07926>.
- [58] Le Yu, Leilei Sun, Bowen Du, and Weifeng Lv. Towards better dynamic graph learning: New architecture and unified library. *Advances in Neural Information Processing Systems*, 36: 67686–67700, 2023.
- [59] Alessio Gravina, Giulio Lovisotto, Claudio Gallicchio, Davide Bacciu, and Claas Grohnfeldt. Long range propagation on continuous-time dynamic graphs. *arXiv preprint arXiv:2406.02740*, 2024.
- [60] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In Anna Korhonen, David Traum, and Lluís Màrquez, editors, *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1285.
- [61] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [62] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. Sparks of artificial general intelligence: Early experiments with gpt-4, 2023.

- [63] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17*, page 1025–1035, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.
- [64] Thomas N. Kipf and Max Welling. Variational graph auto-encoders, 2016.
- [65] Muhan Zhang and Yixin Chen. Link prediction based on graph neural networks. *Advances in neural information processing systems*, 31, 2018.
- [66] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '18*, page 974–983. ACM, July 2018. doi: 10.1145/3219819.3219890.
- [67] Xiang Wang, Xiangnan He, Yixin Cao, Meng Liu, and Tat-Seng Chua. Kgat: Knowledge graph attention network for recommendation. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '19*, page 950–958. ACM, July 2019. doi: 10.1145/3292500.3330989.
- [68] Shiwen Wu, Fei Sun, Wentao Zhang, Xu Xie, and Bin Cui. Graph neural networks in recommender systems: A survey. *ACM Comput. Surv.*, 55(5), December 2022. ISSN 0360-0300. doi: 10.1145/3535101.
- [69] Xiaohui Zhang, Yanbo Wang, Xiyuan Wang, and Muhan Zhang. Efficient neural common neighbor for temporal graph link prediction. In *The Fourth Learning on Graphs Conference*, 2025.
- [70] Xiaodong Lu, Leilei Sun, Tongyu Zhu, and Weifeng Lv. Improving temporal link prediction via temporal walk matrix projection. *Advances in Neural Information Processing Systems*, 37: 141153–141182, 2024.
- [71] Lu Wang, Xiaofu Chang, Shuang Li, Yunfei Chu, Hui Li, Wei Zhang, Xiaofeng He, Le Song, Jingren Zhou, and Hongxia Yang. Tcl: Transformer-based dynamic graph modelling via contrastive learning, 2021.
- [72] Weilin Cong, Si Zhang, Jian Kang, Baichuan Yuan, Hao Wu, Xin Zhou, Hanghang Tong, and Mehrdad Mahdavi. Do we really need complicated model architectures for temporal networks? In *The Eleventh International Conference on Learning Representations*, 2023.
- [73] Yuxi Liu, Lianghao Xia, and Chao Huang. Selfgnn: Self-supervised graph neural networks for sequential recommendation. *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024.
- [74] Anushka Tiwari, Haimonti Dutta, and Shahrzad Khanizadeh. Heterogeneous sequel-aware graph neural networks for sequential learning. *ArXiv*, abs/2506.05625, 2025.
- [75] Maya Bechler-Speicher, Ben Finkelshtein, Fabrizio Frasca, Luis Müller, Jan Tönshoff, Antoine Siraudin, Viktor Zaverkin, Michael M. Bronstein, Mathias Niepert, Bryan Perozzi, Mikhail Galkin, and Christopher Morris. Position: Graph learning will lose relevance due to poor benchmarks. In *Forty-second International Conference on Machine Learning Position Paper Track*, 2025.
- [76] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, Xiaojun Chang, and Chengqi Zhang. Connecting the Dots: Multivariate Time Series Forecasting with Graph Neural Networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '20*, pages 753–763, New York, NY, USA, August 2020. Association for Computing Machinery. ISBN 978-1-4503-7998-4. doi: 10.1145/3394486.3403118.
- [77] Chaoyue Ding, Shiliang Sun, and Jing Zhao. Mst-gat: A multimodal spatial-temporal graph attention network for time series anomaly detection. *Information Fusion*, 89:527–536, 2023. ISSN 1566-2535. doi: <https://doi.org/10.1016/j.inffus.2022.08.011>.

- [78] Ahmad Hafez, Atulya Praphul, Yousef Jaradt, and Ezani Godwin. ConvdySAT: Deep neural representation learning on dynamic graphs via self-attention and convolutional neural networks, 2021.
- [79] L. O. Bentsen, N. Warakagoda, R. Stenbro, and P. Engelstad. It is all connected: A new graph formulation for spatio-temporal forecasting. *ArXiv*, abs/2303.13177, 2023.
- [80] Wei Fan, Jingru Fei, Dingyu Guo, Kun Yi, Xiaozhuang Song, Haolong Xiang, Hangting Ye, and Min Li. Towards multi-resolution spatiotemporal graph learning for medical time series classification. *Proceedings of the ACM on Web Conference 2025*, 2025.
- [81] Zhikai Chen, Haitao Mao, Hang Li, Wei Jin, Haifang Wen, Xiaochi Wei, Shuaiqiang Wang, Dawei Yin, Wenqi Fan, Hui Liu, and Jiliang Tang. Exploring the potential of large language models (llms) in learning on graphs. *ACM SIGKDD Explorations Newsletter*, 25:42 – 61, 2023.
- [82] Bryan Perozzi, Bahare Fatemi, Dustin Zelle, Anton Tsitsulin, Mehran Kazemi, Rami Al-Rfou, and Jonathan J. Halcrow. Let your graph do the talking: Encoding structured data for llms. *ArXiv*, abs/2402.05862, 2024.
- [83] Yijun Tian, Huan Song, Zichen Wang, Haozhu Wang, Ziqing Hu, Fang Wang, N. Chawla, and Panpan Xu. Graph neural prompting with large language models. In *AAAI Conference on Artificial Intelligence*, 2023.
- [84] Yuxiang Wang, Xinnan Dai, Wenqi Fan, and Yao Ma. Exploring graph tasks with pure llms: A comprehensive benchmark and investigation. *ArXiv*, abs/2502.18771, 2025.
- [85] Ruosong Ye, Caiqi Zhang, Runhui Wang, Shuyuan Xu, and Yongfeng Zhang. Language is all a graph needs. In Yvette Graham and Matthew Purver, editors, *Findings of the Association for Computational Linguistics: EACL 2024*, pages 1955–1973, St. Julian’s, Malta, March 2024. Association for Computational Linguistics.
- [86] Zirui Guo, Lianghao Xia, Yanhua Yu, Yuling Wang, Zixuan Yang, Wei Wei, Liang Pang, Tat-Seng Chua, and Chao Huang. Graphedit: Large language models for graph structure learning. *ArXiv*, abs/2402.15183, 2024.
- [87] Yanchao Tan, Hang Lv, Pengxiang Zhan, Shiping Wang, and Carl Yang. Graph-oriented instruction tuning of large language models for generic graph mining. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [88] Xubin Ren, Jiabin Tang, Dawei Yin, Nitesh V. Chawla, and Chao Huang. A survey of large language models for graphs. *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024.
- [89] Chao Huang, Xubin Ren, Jiabin Tang, Dawei Yin, and Nitesh V. Chawla. Large language models for graphs: Progresses and directions. *Companion Proceedings of the ACM Web Conference 2024*, 2024.
- [90] Jiajin Liu, Dongzhe Fan, Jiacheng Shen, Chuanhao Ji, D. Zha, and Qiaoyu Tan. Graph-mllm: Harnessing multimodal large language models for multimodal graph learning. *ArXiv*, abs/2506.10282, 2025.
- [91] Huachi Zhou, Jiahe Du, Chuang Zhou, Chang Yang, Yilin Xiao, Yuxuan Xie, and Xiao Huang. Each graph is a new language: Graph learning with llms. In *Annual Meeting of the Association for Computational Linguistics*, 2025.

A Additional Related Works

A.1 Sequence Modeling

Transformer-based sequence models have delivered state-of-the-art performance across a wide range of language and vision tasks by learning contextual representations over token sequences [38, 60–62]. Given an input sequence, these models map each token to a continuous embedding and use self-attention to capture local and long-range dependencies within the sequence. While highly effective for modeling the internal structure of a single sequence, this paradigm typically treats each sequence in isolation and does not leverage information from *related* sequences.

A.2 Graph Modeling

Graph models capture relational structures among entities (nodes). In particular, graph neural networks (GNNs) propagate and aggregate information over local neighborhoods to learn node, edge, or graph level embeddings [36, 37, 47]. These methods have demonstrated strong performance across tasks such as node classification [36, 63], link prediction [64, 65], and recommendation [66–68]. Standard GNNs model each node in the graph as a single feature vector and do not account for sequential information within nodes. Bridging graph-based relational modeling with sequence-aware representations is therefore an important step toward richer models that capture both intra-sequence patterns and inter-sequence relations.

A.3 Temporal Graph Methods

Temporal graph models such as TGN [27], TGAT [28], DyRep [29], TNCN [69], and others [40, 42, 70–74], attach timestamps to edges to model interactions without aligned time steps. However, a temporal graph requires all events to be described as a relation between two entities. Personal events that involve only a single user cannot be easily represented by a temporal graph. A common workaround is to introduce event nodes (e.g., representing “login” or “payment declined” as nodes) and connect them to the corresponding entities, but this can create high-degree hubs (e.g., a single “login” node connected to many users) and is often unnatural in a graph formulation. As argued by Bechler-Speicher et al. [75], forcing an unsuitable data modality into a graph form can fail to encode important information and may not be meaningful.

A.4 Spatial-Temporal Graph Methods

Spatial-temporal graph networks were originally designed for time-series prediction problems such as traffic forecasting. Models like STGCN [45], STGformer [46], MTGCN [76], and MST-GAT [77], among others [78–80], encode spatial relations (e.g., road networks) and temporal patterns. In recommendation systems, hybrid approaches have also emerged. Shui et al. [32] models the relational graph among items while simultaneously capturing user consumption sequences. Similarly, Zhang et al. [33] treats users and items as different node types within a heterogeneous graph, enabling message passing between items (sequential) and users (relational). A common limitation of these methods is the assumption that all time series are synchronized and share identical time steps. By contrast, our setting imposes no such constraint: sequences may be asynchronous, vary in length, and even be non-overlapping. This flexibility better reflects real-world scenarios where user interactions occur at irregular intervals and exhibit diverse temporal patterns across different users.

A.5 Graph Large Language Models

An emerging line of research seeks to leverage the generalization capabilities of foundation models for graph learning. *Graph large language models (Graph LLMs)* achieve this by integrating pretrained LLMs with graph-structured data through prompting [81–83], instruction tuning [84–87], or hybrid architectures [88–90]. The promise of Graph LLMs lies in bridging reasoning over relational data with the flexibility of natural language interfaces. In line with this view, Zhou et al. [91] even conceptualizes that each graph can be seen as a new language for the LLMs to learn. Our approach, however, differs: rather than adapting pretrained LLMs to graphs, we introduce a unified architecture that integrates sequence modeling with GNNs to jointly capture temporal dynamics and relational structure.

B Algorithms

In this section, we provide the detailed procedural steps for the BRIDGE framework. Algorithm 1 outlines the end-to-end training procedure, illustrating the joint optimization of the sequence model and the graph module.

Algorithm 1 BRIDGE

Require: Sequences $\{S_i\}_{i=1}^N$; Graph $G = (\mathcal{V}, \mathcal{E})$; number of sequence model layers L_s ; number of graph layers L_g

Ensure: Final user representations $\{h_i\}_{i=1}^N$

```

1: for all  $v_i \in \mathcal{V}$  do
2:    $X_i^{(0)} \leftarrow \text{TokenEmbedding}(S_i)$ 
3: end for
4: for all  $\ell = 1, \dots, L_s$  do
5:   for all  $v_i \in \mathcal{V}$  do
6:      $X_i^{(\ell)} \leftarrow \text{TransformerLayer}^{(\ell)}(X_i^{(\ell-1)})$ 
7:   end for
8: end for
9: for all  $v_i \in \mathcal{V}$  do
10:   $Z_i \leftarrow X_i^{(L_s)}$ 
11:   $z_i^{(0)} \leftarrow \text{Compress}(Z_i)$ 
12: end for
13: for all  $\ell = 1, \dots, L_g$  do
14:   for all  $v_i \in \mathcal{V}$  do
15:     $z_i^{(\ell)} \leftarrow \text{MessagePassing}^{(\ell)}(i, z_i^{(\ell-1)}, \{z_j^{(\ell-1)}\}_{j \in \mathcal{N}(i)})$ 
16:   end for
17: end for
18: for all  $v_i \in \mathcal{V}$  do
19:   $h_i \leftarrow z_i^{(L_g)} + \text{Compress}(Z_i)$  {residual}
20: end for

```

C Experimental Details

C.1 Hyperparameters

Relationship prediction In Tables 6 to 9, we report the hyperparameters used for the relationship prediction experiments. We use the following notation: LR denotes the learning rate, and BS denotes the batch size. For methods that use user-sequence embeddings (Two-Stage and BRIDGE), Bert specifies the configuration of the *BERT-style sequence encoder* used to produce user embeddings from event sequences. We report Bert as $L_{\text{TF}} \times H$, where L_{TF} is the number of Transformer encoder layers and H is the number of self-attention heads per layer; for example, 1x2 denotes $L_{\text{TF}} = 1$ and $H = 2$. Hidden denotes the embedding dimension of this encoder. The resulting embeddings are used as node features in Two-Stage baselines, or as the sequence module within BRIDGE.

Across static graph methods, we follow the training setup of Fathony et al. [26] and use a shared learning rate and batch size (Table 6). For temporal graph baselines, we use bs = 4096 by default, but reduce the batch size for methods that exceed GPU memory under this setting; the per-method batch sizes are reported in Table 8. The remaining hyperparameters follow existing works: TGN, DyRep, and TNCN follow Fathony et al. [26], and GraphMixer, NAT, TCL, and TPNet follow the code of Lu et al. [70].

For BRIDGE, we use a default batch size of 128 (reduced when required by memory constraints) and choose the BERT-style sequence encoder configuration to match the user-embedding encoder used by the Two-Stage static baselines.

Fraud detection. We summarize the hyperparameters for fraud detection in Tables 10 to 12. Across method classes, we tune LR and BS, and, when applicable, attention heads and sequence-encoder

Table 6: **Relationship prediction hyperparameters (graph-only static baselines).** We report the learning rate (LR) and batch size (BS) used for each static graph encoder.

Method	LR	BS
GAT	0.001	4096
GCN	0.001	4096
TFConv	0.001	4096

Table 7: **Relationship prediction hyperparameters (two-stage static + sequence baselines).** We report the learning rate (LR), batch size (BS), and sequence-encoder configuration: “Bert” denotes # Transformer layers $\times$ #attention heads, and “Hidden” is the Transformer hidden size.

Method	LR	BS	Bert	Hidden
GAT + S	0.001	4096	1x2	128
GCN + S	0.001	4096	1x2	128
TFConv + S	0.001	4096	1x2	128

capacity parameters, select the configuration with the best validation performance, and report test results averaged over multiple random seeds.

Static graph methods. Table 10 lists hyperparameters for graph-only and two-stage graph baselines operating on the interaction graph, with or without a text encoder. Here, Heads denotes the number of attention heads for attention-based GNNs (not applicable to GCN).

Two-Stage temporal graph methods. Table 11 reports the selected hyperparameters for Two-Stage temporal graph baselines. These methods use review-text embeddings as precomputed node features; Heads denotes the number of attention heads for models that use attention.

BRIDGE methods. Table 12 reports the per-dataset configurations for end-to-end BRIDGE variants. In addition to LR BS, and Heads, we report the classifier depth (Clf-Layers) and the sequence Transformer capacity as separate columns: TF-Layers, TF-Heads, TF-FFN, and TF-Dropout. All tabulated settings are extracted from the same run artifacts used to produce the main fraud detection results.

C.2 Full Relationship Prediction Results

Metrics and protocol. We evaluate relationship prediction using standard ranking metrics, Mean Reciprocal Rank (MRR) and Hits@ k for $k \in \{1, 3, 5, 10\}$, following established benchmarks [52–54]. For each query, we rank the ground-truth target among 101 candidates (one positive and 100 negatives), where negatives are sampled uniformly according to the benchmark protocol and restricted to non-edges for the query. We report the mean and standard deviation over three independent runs.

Full results. Table 13 reports the complete set of ranking metrics for all methods and datasets. In the main paper, we present MRR and Hits@5 for brevity; the additional Hits@ k values reported here provide a more detailed view of performance across operating points and do not change the qualitative conclusions.

Table 8: **Relationship prediction hyperparameters (temporal graph baselines).** We report the learning rate (LR) and batch size (BS) used for each temporal graph method.

Method	LR	BS
DyRep	0.001	4096
TGN	0.001	4096
TCL	0.001	1024
NAT	0.001	1024
GraphMixer	0.001	1024
TPNet	0.001	1024

Table 9: **Relationship prediction hyperparameters (BRIDGE)**. For each dataset and graph backbone, we report the learning rate (LR), batch size (BS), and sequence-encoder configuration. “Bert” denotes #Transformer layers $\times$ #attention heads, and “Hidden” is the Transformer hidden size. “—” indicates the configuration is not applicable or not run for that dataset/backbone.

Method	Dataset	LR	BS	Bert	Hidden
BRIDGE-GCN	Brightkite	0.0001	128	1x2	128
	Gowalla	0.0001	32	1x2	128
	Amazon’s	0.0001	32	1x2	128
BRIDGE-GAT	Brightkite	0.0001	128	1x2	128
	Gowalla	0.0001	32	1x2	128
	Amazon’s	0.0001	32	1x2	128
BRIDGE-TFConv	Brightkite	0.0001	128	1x2	128
	Gowalla	0.0001	32	1x2	128
	Amazon’s	0.0001	32	1x2	128

Table 10: **Fraud detection hyperparameters (static graph baselines)**. We report the learning rate (LR), batch size (BS), and number of attention heads (Heads) used for each static graph method. “—” indicates the method is non-attentional and does not use heads.

Method	LR	BS	Heads
GAT	0.001	4096	2
GCN	0.001	4096	—
TFConv	0.001	4096	2

696 D Full Ablation Results

697 Tables 14 and 15 and ?? show the full results of ablation experiments. For each variant, we list the
698 test MRR, per-epoch runtime (minutes), and runtime normalized by the reference configuration used
699 in the main experiments (bolded).

700 E Limitations and Discussion

701 While BRIDGE provides a simple way to combine user event histories with static relation graphs,
702 several limitations remain.

703 E.1 Use of Time Information

704 BRIDGE preserves event order but does not explicitly encode absolute timestamps or inter-event time
705 gaps. For tasks where absolute time or time intervals are informative (e.g., periodicity or burstiness),
706 incorporating relative/absolute time or time-aware positional encodings may yield further gains,
707 which we leave to future work.

708 E.2 Labels, Data, and Deployment

709 The fraud detection labels are proxy labels derived from review helpfulness, so they should not be
710 interpreted as verified ground truth fraud labels. The experiments use public datasets, which improves
711 reproducibility but may not capture privacy constraints, fairness requirements, or distribution shifts
712 that arise in deployed user behavior systems. Any deployment would require domain-specific
713 validation and safeguards around sensitive user data.

714 E.3 Computational Complexity

715 A practical limitation of BRIDGE is computational cost when sequences are long or neighborhood
716 fanout is large. We summarize the dominant asymptotic terms to clarify the scalability trade-offs and
717 motivate efficiency improvements.

Table 11: **Fraud detection hyperparameters (temporal graph baselines).** We report the learning rate (LR), batch size (BS), and attention heads (Heads) used in the temporal encoder for each method.

Method	LR	BS	Heads
DyRep	0.001	32768	2
TGN	0.001	32768	2
TCL	0.001	32768	2
NAT	0.001	32768	1
GraphMixer	0.001	32768	1
TPNet	0.001	32768	1

Table 12: **Fraud detection hyperparameters (BRIDGE variants).** For each dataset and Bridge backbone, we list the optimizer learning rate (LR) and batch size (BS), the number of graph attention heads (Heads; “–” for non-attentional GCN), the number of classifier MLP layers (Clf-Layers), and Transformer sequence-encoder settings (TF-Layers/Heads, FFN hidden size TF-FFN, and dropout TF-Dropout).

Method	Dataset	LR	BS	Heads	Clf.	TF-L	TF-H	FFN	Drop.
GAT+GAT	Movies	0.0003	512	1	2	2	4	512	0.2
	Electronics	0.0003	512	1	2	2	4	256	0.2
	Clothing	0.0003	512	1	2	2	2	256	0.2
GCN+GCN	Movies	0.0003	512	–	2	2	4	512	0.2
	Electronics	0.0003	512	–	2	2	4	256	0.2
	Clothing	0.0003	512	–	2	2	2	256	0.15
TFConv+TFConv	Movies	0.0003	512	1	2	2	4	512	0.2
	Electronics	0.0003	512	1	2	2	4	256	0.2
	Clothing	0.0003	512	1	2	2	2	256	0.15

For each node $v_i \in \mathcal{V}$ with event sequence S_i of length M_i , the sequence encoder produces token embeddings $X_i \in \mathbb{R}^{M_i \times d}$. We analyze the Transformer encoder used in our main experiments, and quantify the computation cost of the graph layer in BRIDGE-GCN.

Sequence encoding (Transformer). For a single Transformer layer on a length- M_i sequence, the dominant cost comes from self-attention and the feed-forward blocks:

$$\mathcal{O}_{\text{TF}}(i) = \mathcal{O}(M_i^2 d + M_i d^2). \quad (1)$$

The $M_i^2 d$ term corresponds to attention score computation and attention-weighted value aggregation, while $M_i d^2$ accounts for projections and MLPs. Over all N nodes, the total sequence-encoding cost is

$$\sum_{i=1}^N \mathcal{O}_{\text{TF}}(i) = \mathcal{O}\left(\sum_{i=1}^N (M_i^2 d + M_i d^2)\right). \quad (2)$$

Although quadratic in M_i , Transformer encoding is highly parallelizable across tokens and across nodes.

Where the cost is paid in BRIDGE. The Transformer encoder is applied independently per node to obtain $\{X_i\}$. In BRIDGE-GCN (and other vector-feature GNN backbones), X_i is compressed to a single vector before graph propagation; thus, the $M_i \times d$ feature maps need not be retained throughout message passing.

Computation cost in the graph module. To isolate the graph-phase overhead (holding the same batch size B , fanout k , and hidden size d), BRIDGE-GCN compresses sequences before propagation and performs message passing on d -dimensional vectors, yielding

$$\mathcal{O}_{\text{GCN}} = \mathcal{O}(B k d^2). \quad (3)$$

BRIDGE-GCN decouples graph propagation from sequence length, making it scalable for large graphs or long sequences.

Table 13: **Relationship prediction ranking performance on Brightkite and Gowalla (higher is better)**. We report mean $\pm$ std over three seeds. **Hits@k** is the fraction of test queries for which at least one ground-truth relation appears in the top- k predictions. Best results are **bolded** and runner-ups are underlined.

Method	Brightkite					Gowalla				
	MRR	H@1	H@3	H@5	H@10	MRR	H@1	H@3	H@5	H@10
<i>Graph Only (Static)</i>										
GCN	67.4 $\pm$ 1.9	55.6 $\pm$ 2.4	75.5 $\pm$ 1.8	82.3 $\pm$ 1.3	89.2 $\pm$ 0.7	77.8 $\pm$ 0.6	68.6 $\pm$ 0.8	84.9 $\pm$ 0.4	89.4 $\pm$ 0.3	93.5 $\pm$ 0.2
GAT	65.4 $\pm$ 1.6	53.6 $\pm$ 2.0	73.0 $\pm$ 1.4	80.1 $\pm$ 1.0	87.8 $\pm$ 0.6	78.1 $\pm$ 0.6	69.2 $\pm$ 0.9	84.9 $\pm$ 0.4	89.3 $\pm$ 0.3	93.3 $\pm$ 0.3
TFCnv	68.5 $\pm$ 0.2	56.9 $\pm$ 0.2	76.6 $\pm$ 0.4	83.2 $\pm$ 0.4	89.8 $\pm$ 0.3	80.5 $\pm$ 0.2	72.3 $\pm$ 0.2	87.0 $\pm$ 0.2	90.9 $\pm$ 0.2	94.3 $\pm$ 0.1
<i>Two-Stage (Static + Sequence)</i>										
GCN + S	68.6 $\pm$ 1.1	56.9 $\pm$ 1.4	76.7 $\pm$ 1.0	83.4 $\pm$ 0.7	90.0 $\pm$ 0.3	79.8 $\pm$ 0.7	71.3 $\pm$ 0.9	86.5 $\pm$ 0.5	90.5 $\pm$ 0.4	94.1 $\pm$ 0.4
GAT + S	67.9 $\pm$ 0.5	56.4 $\pm$ 0.6	75.7 $\pm$ 0.6	82.3 $\pm$ 0.5	89.4 $\pm$ 0.2	80.0 $\pm$ 0.7	71.7 $\pm$ 0.9	86.3 $\pm$ 0.6	90.3 $\pm$ 0.5	93.9 $\pm$ 0.5
TFCnv + S	72.7 $\pm$ 0.3	61.9 $\pm$ 0.4	80.5 $\pm$ 0.3	86.3 $\pm$ 0.1	91.7 $\pm$ 0.1	82.5 $\pm$ 0.5	74.8 $\pm$ 0.7	88.6 $\pm$ 0.4	92.0 $\pm$ 0.3	95.0 $\pm$ 0.2
<i>Temporal Graphs</i>										
DyRep	26.4 $\pm$ 3.7	14.0 $\pm$ 3.1	28.2 $\pm$ 4.8	37.5 $\pm$ 5.5	53.4 $\pm$ 5.5	27.8 $\pm$ 2.1	16.6 $\pm$ 1.3	29.6 $\pm$ 2.5	37.8 $\pm$ 3.3	51.2 $\pm$ 4.6
TGN	38.7 $\pm$ 1.6	24.8 $\pm$ 1.8	43.8 $\pm$ 1.7	53.9 $\pm$ 1.3	68.1 $\pm$ 1.1	44.8 $\pm$ 2.7	31.8 $\pm$ 2.6	50.2 $\pm$ 3.4	59.3 $\pm$ 3.4	71.9 $\pm$ 2.7
TCL	20.4 $\pm$ 1.0	9.9 $\pm$ 0.7	21.2 $\pm$ 1.2	29.1 $\pm$ 1.5	43.0 $\pm$ 2.0	20.5 $\pm$ 1.7	11.1 $\pm$ 1.3	20.4 $\pm$ 2.1	27.5 $\pm$ 2.5	39.9 $\pm$ 3.2
NAT	68.2 $\pm$ 7.8	60.3 $\pm$ 9.7	71.9 $\pm$ 7.3	77.0 $\pm$ 5.9	83.9 $\pm$ 3.7	59.1 $\pm$ 13.0	50.6 $\pm$ 14.7	61.8 $\pm$ 13.5	68.0 $\pm$ 11.8	77.0 $\pm$ 8.5
GraphMixer	26.0 $\pm$ 0.5	8.9 $\pm$ 0.6	21.6 $\pm$ 0.6	31.0 $\pm$ 0.4	46.5 $\pm$ 0.5	24.9 $\pm$ 0.5	13.3 $\pm$ 0.4	26.5 $\pm$ 0.6	35.5 $\pm$ 0.7	50.1 $\pm$ 0.7
TPNet	45.0 $\pm$ 0.6	35.7 $\pm$ 0.8	47.0 $\pm$ 0.5	53.6 $\pm$ 0.4	64.3 $\pm$ 0.4	51.4 $\pm$ 1.0	40.8 $\pm$ 0.8	54.8 $\pm$ 1.2	62.9 $\pm$ 1.3	74.4 $\pm$ 1.5
TNCN	53.8 $\pm$ 0.5	42.9 $\pm$ 0.4	58.2 $\pm$ 0.6	65.9 $\pm$ 1.0	76.0 $\pm$ 1.1	51.8 $\pm$ 0.7	40.7 $\pm$ 0.6	55.9 $\pm$ 0.9	64.0 $\pm$ 0.8	75.4 $\pm$ 0.8
BRIDGE										
BRIDGE-GCN	92.2 $\pm$ 0.6	89.9 $\pm$ 0.7	93.6 $\pm$ 0.6	94.8 $\pm$ 0.5	96.3 $\pm$ 0.4	87.9$\pm$0.2	83.4$\pm$0.3	91.1$\pm$0.1	93.3$\pm$0.1	95.5$\pm$0.1
BRIDGE-GAT	78.8 $\pm$ 0.4	70.2 $\pm$ 0.5	85.3 $\pm$ 0.3	89.5 $\pm$ 0.1	93.4 $\pm$ 0.1	<u>83.4$\pm$0.1</u>	<u>76.3$\pm$0.2</u>	<u>89.0$\pm$0.0</u>	<u>92.2$\pm$0.1</u>	<u>95.2$\pm$0.0</u>
BRIDGE-TFCnv	78.4 $\pm$ 0.7	69.6 $\pm$ 1.0	84.9 $\pm$ 0.7	89.3 $\pm$ 0.4	93.4 $\pm$ 0.3	82.9 $\pm$ 0.1	75.5 $\pm$ 0.1	88.7 $\pm$ 0.1	92.1 $\pm$ 0.1	95.1 $\pm$ 0.1

Table 13: **Relationship prediction ranking performance on Amazon-Clothing and Amazon-Electronics (higher is better)**. Mean $\pm$ std over three seeds; best bolded, runner-up underlined.

Method	Clothing					Electronics				
	MRR	H@1	H@3	H@5	H@10	MRR	H@1	H@3	H@5	H@10
<i>Graph Only (Static)</i>										
GCN	17.3 $\pm$ 1.6	9.1 $\pm$ 1.7	17.4 $\pm$ 2.0	23.4 $\pm$ 1.9	34.1 $\pm$ 0.7	48.9 $\pm$ 3.9	34.9 $\pm$ 4.4	56.9 $\pm$ 4.2	66.6 $\pm$ 3.3	76.5 $\pm$ 2.2
GAT	15.9 $\pm$ 1.9	9.9 $\pm$ 2.0	14.0 $\pm$ 2.5	17.9 $\pm$ 2.1	27.5 $\pm$ 1.5	44.7 $\pm$ 0.7	30.5 $\pm$ 0.8	51.8 $\pm$ 0.8	62.5 $\pm$ 0.7	74.6 $\pm$ 0.4
TFCnv	18.8 $\pm$ 0.5	10.4 $\pm$ 0.9	19.1 $\pm$ 0.2	25.0 $\pm$ 0.4	35.6 $\pm$ 0.2	55.8 $\pm$ 0.7	43.0 $\pm$ 1.0	63.8 $\pm$ 0.6	71.5 $\pm$ 0.4	79.5 $\pm$ 0.1
<i>Two-Stage (Static + Sequence)</i>										
GCN + S	11.3 $\pm$ 1.5	3.1 $\pm$ 1.1	9.7 $\pm$ 2.9	15.6 $\pm$ 2.8	30.1 $\pm$ 1.5	50.8 $\pm$ 0.8	37.1 $\pm$ 0.9	58.8 $\pm$ 1.1	68.1 $\pm$ 0.9	77.2 $\pm$ 0.5
GAT + S	11.7 $\pm$ 3.5	5.6 $\pm$ 3.5	9.3 $\pm$ 4.5	12.7 $\pm$ 4.5	23.3 $\pm$ 2.9	50.3 $\pm$ 0.3	36.6 $\pm$ 0.3	58.1 $\pm$ 0.9	67.2 $\pm$ 0.4	76.7 $\pm$ 0.3
TFCnv + S	19.8 $\pm$ 2.0	10.3 $\pm$ 1.5	20.2 $\pm$ 2.9	27.5 $\pm$ 3.4	39.6 $\pm$ 4.3	58.4 $\pm$ 0.2	46.2 $\pm$ 0.3	66.2 $\pm$ 0.1	73.5 $\pm$ 0.1	80.8 $\pm$ 0.0
<i>Temporal Graphs</i>										
DyRep	13.3 $\pm$ 0.8	3.6 $\pm$ 0.6	11.2 $\pm$ 1.0	18.9 $\pm$ 1.3	34.3 $\pm$ 1.5	17.0 $\pm$ 1.9	7.0 $\pm$ 1.8	16.8 $\pm$ 2.4	23.7 $\pm$ 2.1	37.0 $\pm$ 2.1
TGN	14.8 $\pm$ 1.4	4.9 $\pm$ 0.8	13.4 $\pm$ 2.1	20.8 $\pm$ 3.0	36.9 $\pm$ 3.4	33.8 $\pm$ 1.5	21.1 $\pm$ 1.3	37.6 $\pm$ 1.8	46.5 $\pm$ 2.1	60.0 $\pm$ 1.8
TCL	14.2 $\pm$ 0.5	5.3 $\pm$ 0.2	13.6 $\pm$ 1.1	20.5 $\pm$ 1.3	32.9 $\pm$ 1.5	34.9 $\pm$ 0.2	22.4 $\pm$ 0.3	39.2 $\pm$ 0.1	47.3 $\pm$ 0.1	59.6 $\pm$ 0.2
NAT	30.6 $\pm$ 7.9	23.3 $\pm$ 9.1	32.2 $\pm$ 8.3	36.8 $\pm$ 7.2	43.5 $\pm$ 4.1	62.8 $\pm$ 8.0	55.7 $\pm$ 9.7	65.6 $\pm$ 8.0	70.4 $\pm$ 6.4	77.0 $\pm$ 3.7
GraphMixer	12.9 $\pm$ 0.3	4.6 $\pm$ 0.3	11.7 $\pm$ 0.3	17.7 $\pm$ 0.4	30.1 $\pm$ 0.8	23.1 $\pm$ 0.5	10.4 $\pm$ 0.5	24.3 $\pm$ 0.5	34.3 $\pm$ 0.5	51.8 $\pm$ 0.5
TPNet	11.4 $\pm$ 0.6	3.5 $\pm$ 0.3	9.1 $\pm$ 0.8	14.8 $\pm$ 1.1	27.6 $\pm$ 1.3	22.9 $\pm$ 0.2	9.9 $\pm$ 0.3	24.2 $\pm$ 0.3	34.7 $\pm$ 0.3	52.9 $\pm$ 0.5
TNCN	31.7 $\pm$ 1.8	22.5 $\pm$ 1.7	33.9 $\pm$ 2.5	39.6 $\pm$ 2.1	48.1 $\pm$ 1.6	45.4 $\pm$ 1.1	35.7 $\pm$ 1.2	49.8 $\pm$ 1.1	55.8 $\pm$ 0.9	63.0 $\pm$ 1.2
BRIDGE										
BRIDGE-GCN	52.3 $\pm$ 2.9	37.1 $\pm$ 4.4	62.5 $\pm$ 2.7	72.5 $\pm$ 2.2	80.6$\pm$1.7	80.2$\pm$0.2	74.5$\pm$0.4	84.3$\pm$0.1	87.1$\pm$0.1	89.9$\pm$0.2
BRIDGE-GAT	63.7 $\pm$ 0.8	54.4 $\pm$ 1.1	70.7 $\pm$ 0.4	75.2$\pm$0.2	78.9 $\pm$ 0.6	75.4 $\pm$ 0.3	66.9 $\pm$ 0.4	<u>82.2$\pm$0.1</u>	<u>86.0$\pm$0.1</u>	89.0 $\pm$ 0.1
BRIDGE-TFCnv	54.0 $\pm$ 0.5	42.0 $\pm$ 0.8	59.7 $\pm$ 0.7	68.0 $\pm$ 0.7	<u>79.2$\pm$1.0</u>	75.1 $\pm$ 0.2	66.4 $\pm$ 0.2	81.7 $\pm$ 0.3	85.7 $\pm$ 0.2	<u>89.3$\pm$0.0</u>

Table 13: **Relationship prediction ranking performance on Amazon-Movies (higher is better).** Mean $\pm$ std over three seeds; best bolded, runner-up underlined.

Method	Amazon-Movies				
	MRR	Hits@1	Hits@3	Hits@5	Hits@10
<i>Graph Only (Static)</i>					
GCN	76.4 $\pm$ 0.5	65.8 $\pm$ 0.6	84.5 $\pm$ 0.7	89.6 $\pm$ 0.5	94.6 $\pm$ 0.2
GAT	76.6 $\pm$ 0.7	65.5 $\pm$ 0.8	85.5 $\pm$ 0.8	90.8 $\pm$ 0.6	95.3 $\pm$ 0.3
TFCnv	82.4 $\pm$ 0.1	73.0 $\pm$ 0.2	90.5 $\pm$ 0.0	94.4 $\pm$ 0.0	97.4 $\pm$ 0.0
<i>Two-Stage (Static + Sequence)</i>					
GCN + S	76.8 $\pm$ 1.1	66.3 $\pm$ 1.4	84.9 $\pm$ 0.9	90.1 $\pm$ 0.6	94.9 $\pm$ 0.3
GAT + S	77.2 $\pm$ 0.5	66.6 $\pm$ 0.7	85.7 $\pm$ 0.5	90.9 $\pm$ 0.4	95.4 $\pm$ 0.3
TFCnv + S	83.6 $\pm$ 0.1	74.5 $\pm$ 0.1	91.5 $\pm$ 0.2	95.1 $\pm$ 0.1	97.7 $\pm$ 0.0
<i>Temporal Graphs</i>					
DyRep	41.6 $\pm$ 3.8	26.9 $\pm$ 4.4	47.8 $\pm$ 4.0	58.4 $\pm$ 3.2	72.5 $\pm$ 2.0
TGN	59.1 $\pm$ 3.2	44.5 $\pm$ 4.0	68.9 $\pm$ 3.1	77.6 $\pm$ 2.1	86.1 $\pm$ 1.0
TCL	72.2 $\pm$ 1.4	60.0 $\pm$ 1.8	81.5 $\pm$ 1.2	87.6 $\pm$ 0.8	93.5 $\pm$ 0.5
NAT	78.8 $\pm$ 4.0	69.7 $\pm$ 5.1	85.5 $\pm$ 3.8	90.7 $\pm$ 2.7	95.4 $\pm$ 1.3
GraphMixer	55.0 $\pm$ 0.8	40.8 $\pm$ 1.1	63.2 $\pm$ 1.1	72.5 $\pm$ 0.8	83.2 $\pm$ 0.5
TPNet	42.5 $\pm$ 0.9	27.5 $\pm$ 0.7	51.4 $\pm$ 1.2	61.6 $\pm$ 1.2	71.8 $\pm$ 0.9
TNCN	20.8 $\pm$ 1.2	11.8 $\pm$ 1.0	22.0 $\pm$ 1.8	28.6 $\pm$ 2.1	39.2 $\pm$ 1.7
BRIDGE					
BRIDGE-GCN	90.4$\pm$0.4	84.7$\pm$0.6	95.6$\pm$0.2	97.6$\pm$0.2	99.0$\pm$0.1
BRIDGE-GAT	89.1 $\pm$ 0.0	82.4 $\pm$ 0.1	95.2 $\pm$ 0.0	97.4 $\pm$ 0.0	98.9 $\pm$ 0.0
BRIDGE-TFCnv	88.6 $\pm$ 0.4	81.8 $\pm$ 0.6	94.9 $\pm$ 0.2	97.2 $\pm$ 0.1	98.7 $\pm$ 0.1

Table 14: **Sequence module depth.** Sensitivity to the number of Transformer layers in the sequence encoder. We report test MRR (mean $\pm$ std over three seeds) and per-epoch runtime (minutes); **Rel. Epoch Time** is normalized to the two-layer reference used in this sweep.

# TF Layers	MRR	Epoch Time (min)	Rel. Epoch Time
1	62.4 $\pm$ 0.4	43.8 $\pm$ 0.3	0.96
2	62.4 $\pm$ 0.4	45.7 $\pm$ 0.1	1.00
4	62.5 $\pm$ 0.3	49.5 $\pm$ 0.1	1.08

Table 15: Effect of neighborhood fanout on the full Brightkite friend-recommendation task. Mean and standard deviation are over completed seeds.

# Neighbors	MRR	Time (min)	Rel. Time
2	77.3 $\pm$ 0.6	301.4 $\pm$ 120.4	0.09
5	84.2 $\pm$ 0.4	1009.7 $\pm$ 96.1	0.31
10	85.4 $\pm$ 0.6	3264.4 $\pm$ 770.9	1.00
15	84.4 $\pm$ 0.3	4202.2 $\pm$ 609.0	1.29

Table 16: Effect of relational message passing on the full Brightkite friend-recommendation task. Mean and standard deviation are over completed seeds.

Variant	MRR	Time (min)	Rel. Time
With relational MP	85.4 $\pm$ 0.6	3264.4 $\pm$ 770.9	1.00
No relational MP	62.0 $\pm$ 0.2	93.9 $\pm$ 1.0	0.03

739

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes].

Justification: The abstract and Section 1 scope the work to user behavior modeling with event histories and static relations, and the claims are supported by the experiments in Section 4.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes].

Justification: Appendix E provides discussion on the limitations, including lack of absolute-time modeling, proxy fraud labels, and deployment concerns; .

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A].

Justification: The paper is empirical and architectural; it does not present theoretical results or proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes].

Justification: Section 4 describes datasets, baselines, metrics, and evaluation protocol, while Appendix C reports hyperparameters and full results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes].

Justification: We provide our code along with the paper submission

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes].

Justification: Section 4 specifies the experimental protocols and metrics, and Appendix C reports the selected hyperparameters for the baselines and BRIDGE variants.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes].

Justification: The result tables report mean and standard deviation over multiple random seeds; the metric definitions and reporting protocol are described in Section 4 and Appendix C.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).

- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes].

Justification: The paper reports runtime trade-offs for ablations and discusses computational complexity and the hardware we used.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes].

Justification: The work uses cited public datasets and does not involve deployment or collection of new sensitive user data; limitations and deployment risks are discussed in Section 6 and Appendix E.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes].

Justification: The limitations discussion notes possible deployment concerns around privacy, fairness, and distribution shift for user behavior systems.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.

- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A].

Justification: The paper does not release a high-risk model or newly scraped dataset; it evaluates methods on cited public datasets.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes].

Justification: The datasets and pretrained text encoder are credited through citations.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.

- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

1011 13. New assets

1012 Question: Are new assets introduced in the paper well documented and is the documentation
1013 provided alongside the assets?

1014 Answer: [N/A].

1015 Justification: The paper does not introduce a new dataset, benchmark, or released model
1016 asset.

1017 Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

1026 14. Crowdsourcing and research with human subjects

1027 Question: For crowdsourcing experiments and research with human subjects, does the paper
1028 include the full text of instructions given to participants and screenshots, if applicable, as
1029 well as details about compensation (if any)?

1030 Answer: [N/A].

1031 Justification: The paper does not conduct crowdsourcing or new human-subject experiments.

1032 Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

1041 15. Institutional review board (IRB) approvals or equivalent for research with human 1042 subjects

1043 Question: Does the paper describe potential risks incurred by study participants, whether
1044 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
1045 approvals (or an equivalent approval/review based on the requirements of your country or
1046 institution) were obtained?

1047 Answer: [N/A].

1048 Justification: The paper uses existing public datasets and does not conduct new human-
1049 subject research.

1050 Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- 1053 • Depending on the country in which research is conducted, IRB approval (or equivalent)
1054 may be required for any human subjects research. If you obtained IRB approval, you
1055 should clearly state this in the paper.
- 1056 • We recognize that the procedures for this may vary significantly between institutions
1057 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
1058 guidelines for their institution.
- 1059 • For initial submissions, do not include any information that would break anonymity (if
1060 applicable), such as the institution conducting the review.

1061 16. **Declaration of LLM usage**

1062 Question: Does the paper describe the usage of LLMs if it is an important, original, or
1063 non-standard component of the core methods in this research? Note that if the LLM is used
1064 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
1065 scientific rigor, or originality of the research, declaration is not required.

1066 Answer: [N/A].

1067 Justification: LLMs are not an important, original, or non-standard component of the
1068 proposed method or experiments.

1069 Guidelines:

- 1070 • The answer [N/A] means that the core method development in this research does not
1071 involve LLMs as any important, original, or non-standard components.
- 1072 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1073 be described.